

PROJECT REPORT

CLINIC MANAGEMENT & PRESCRIPTION SYSTEM

Course: Java Programming

Candidate Name: Naman Singh

Registration Number: 25BAI11153

1. Abstract

The Clinic Management & Prescription System is a console-based software solution engineered in Core Java to streamline everyday outpatient clinical workflows. The software provides an integrated environment to maintain patient demographics, doctor specialties, time-slot allocation with active conflict detection, and prescription logging. Designed strictly under Object-Oriented Programming (OOP) paradigms, the system functions entirely within volatile memory using Java Collections, eliminating external database dependencies while remaining executable via standard command-line interfaces.

2. Introduction & Problem Statement

Small-to-medium healthcare clinics frequently rely on physical registers or fragmented digital spreadsheets to handle appointment schedules and doctor consultations. This operational approach introduces significant vulnerabilities:

- **Double-Booking Errors:** Manual record-keeping struggles to verify concurrent doctor availability across overlapping time frames.
- **Prescription Disconnect:** Medical prescriptions are often separated from the patient's appointment logs, causing fragmented historical tracking.
- **System Overhead:** Enterprise Hospital Information Systems (HIS) impose excessive computational, network, and setup overhead for standalone consultation desks.

This project addresses these challenges by offering a lightweight, zero-dependency, terminal-based application that enforces scheduling validation rules and links patient visits directly with prescription data.

3. System Requirements & Specifications

Operational Environment:

- **Programming Language:** Java SE (JDK 8 or higher)

- **Execution Interface:** Terminal / Command Line Interface (CLI) (Linux Bash, macOS Terminal, Windows Command Prompt/PowerShell)
- **External Dependencies:** None (Zero external .jar files or database engines required)

Core Libraries Utilized:

- java.util.Scanner: Handles standard console character streaming and user input capture.
- java.util.ArrayList: Supplies dynamically resizable in-memory list implementations for application state tracking.

4. Software Architecture & OOP Implementation

The application adheres to modular design patterns by separating entity models from procedural execution logic across four dedicated class files.

Class	Primary Responsibility	Key Encapsulated Attributes
Patient	Models patient identity and demographic data	id, name, age, phone
Doctor	Represents medical practitioners on roster	id, name, specialization
Appointment	Manages visit slots, statuses, and prescriptions	aptId, patientId, doctorId, time, status, med, dose
Main	Controls the CLI menu, validation loops, and data store	registeredPatients, availableDoctors, scheduledAppointments

Object-Oriented Principles Applied

- **Encapsulation:** Class state variables are declared with private access modifiers to prevent unauthorized external mutation. Access and state updates occur strictly via explicit public getter and setter methods (getId(), setStatus(), givePrescription()).
- **Polymorphism & String Representation:** Domain entities override the native public String toString() method inherited from java.lang.Object, enabling clean, formatted record printing during terminal rendering.
- **Separation of Concerns:** Business logic (such as conflict resolution and validation) resides inside Main, isolating data modeling classes from input-output handling.

5. Algorithmic Workflows & Logic

5.1 Unique Identification Verification

Prior to writing new objects into registeredPatients or availableDoctors, the system performs a linear scan over the active list:

1. Accept candidate identifier string (uniquePatientId / doctorId).
2. Traverse existing records comparing existing keys via String.equals().
3. If an identical key is detected, abort insertion, issue a warning message, and safely return control to the menu.

5.2 Conflict Detection & Slot Booking

The scheduling engine enforces multi-variable validation before an appointment record is committed:

1. **Existence Verification:** Iterates through registeredPatients and availableDoctors to ensure both foreign keys reference registered entities.
2. **Temporal Collision Engine:** Iterates through scheduledAppointments evaluating three concurrent criteria:
3. **Collision** = (Appt.doctorId == targetDoctorId) **AND** (Appt.time == targetTime) **AND** (Appt.status == "Scheduled")
4. If all three evaluate to true, the requested slot is flagged as occupied, preventing overlapping bookings for the same doctor.

5.3 Prescription Allocation & Lifecycle Progression

Appointments follow a defined state transition:

Scheduled —(Prescription Added)—> **Completed**

Scheduled —(Cancellation)—> **Cancelled**

Executing the prescription module locates the target aptId, attaches the medication name and dosage parameters, and marks the appointment as Completed.

6. Verification & Test Scenarios

The system underwent edge-case verification during development:

Test Case	Simulated Input	Expected System Behavior	Result
Duplicate Patient ID	Existing ID P1 supplied during registration	Abort registration; log rejection notice	Passed
Non-Existent Foreign Key	Patient P99 booked with Doctor D1	Abort booking; demand valid patient creation	Passed
Doctor Schedule Collision	Booking D1 at 10:00 AM when already reserved	Detect collision; preserve existing record	Passed
Invalid Numeric Input	Alphabetical characters entered into age prompt	Catch NumberFormatException; prevent crash	Passed
Post-Cancellation Slot Reuse	Booking a slot previously marked Cancelled	Allow booking; ignore non-active records	Passed

7. Conclusion & Future Enhancements

The Clinic Management & Prescription System fulfills all core academic and technical benchmarks for command-line execution, robust OOP encapsulation, and dynamic list traversal. The project operates reliably without GUI overhead or platform-specific prerequisites.

Prospective Enhancements:

- **File Persistence:** Integrating file I/O serialization (java.io) or SQLite to persist records across execution sessions.
- **Calendar Granularity:** Transitioning time slots from arbitrary strings to strict temporal objects using the java.time API.
- **Role-Based Access Control (RBAC):** Introducing authenticated views separating administrative tools from doctor consultation views.